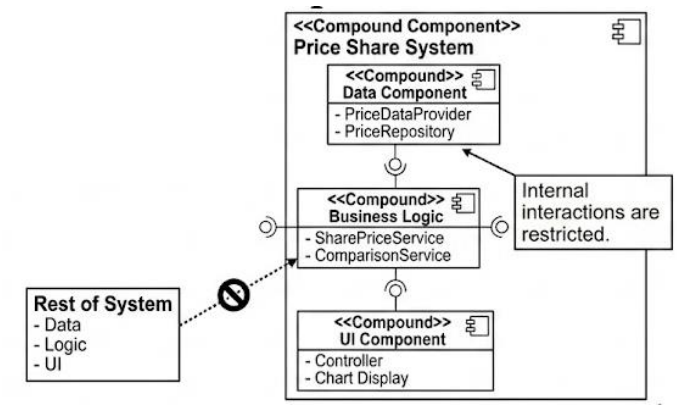
Software Architecture and Design Coursework Sprint 3: Share price
Abbas Mohamed – A00041006
Keeley Sheridan – E23598145

Architecture divided into logical compound components



UI Layer

- **SearchForm:** User inputs stock symbols
- **ComparisonPanel:** manages selected stocks 1 to 2 to compare
- **ChartView:** Displays price charts
- **ErrorBanner:** Shows errors when presented

Service Layer

- **SharePriceService:** Handles comparison, Retrieves and aggravates price history
- **ChartService:** Transforms data into chart ready format

Data Layer

- **Repository:** Stores or Retrieves price history data to a maximum keep of 2 years
- **Provider:** (Yahoo API): External Data source

Business Component

- SharePriceService
- ComparisonService

Responsibility

Processes system logic and coordinates data

Data Component

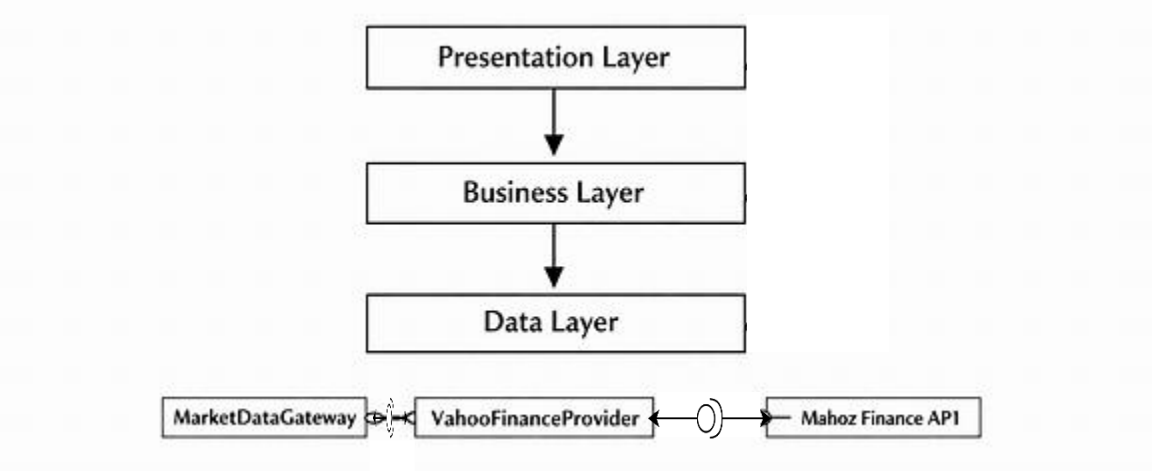
- YahooFinanceProvider

- SQLitePriceRepository
- ChartService

Responsibility

Handles data retrieval, storage, and visualization

Domain-Independent Styles

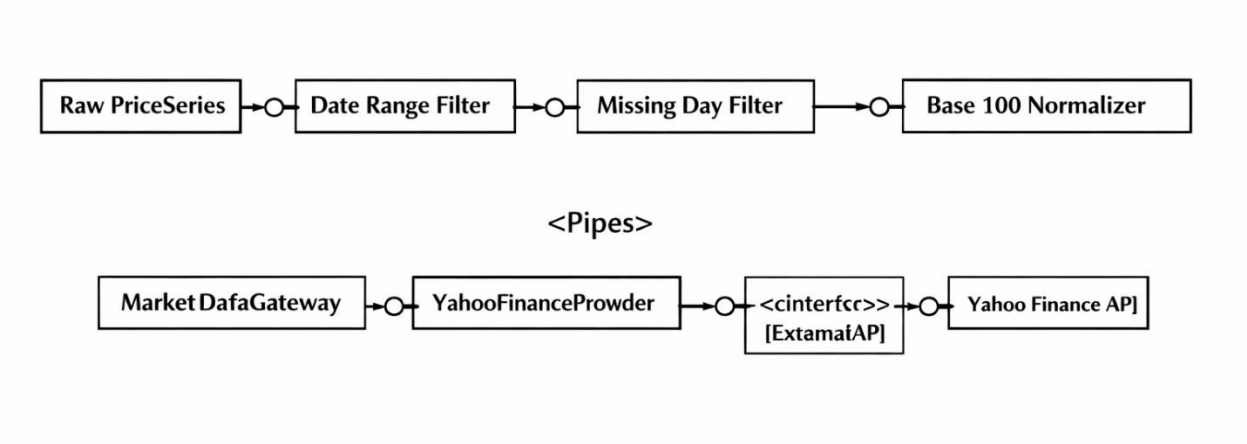


Justification

- Separates concerns clearly
- Improves maintainability
- Supports Clean Architecture

Software Architecture and Design Coursework Sprint 3: Share price
Abbas Mohamed – A00041006
Keeley Sheridan – E23598145

Pipe and Filter



- Raw PriceSeries enters the pipeline.
- Passes through filters: Date Range Filter → Missing Day Filter → Base 100 Normalizer.
- Each filter is independent, connected by pipes that carry data forward.
- The output is a Chart-Ready PriceSeries ready for visualization.

Software Architecture and Design Coursework Sprint 3: Share price
Abbas Mohamed – A00041006
Keeley Sheridan – E23598145

Architectural design

The Share Price Comparison System is designed using a combination of established architectural patterns to ensure scalability, maintainability, and clear separation of concerns. The system structure aligns with layered architecture, MVC, adapter, pipes and filters, and N-tier principles.

The system follows a Layered Architecture, dividing responsibilities into distinct layers: Presentation, Controller, Service, and Data.

The Presentation layer consists of user interface components such as SearchForm, ComparisonPanel, ChartView, and ErrorBanner, which handle user interaction.

The Controller layer (implicit in the design) processes user inputs and delegates actions to the Service layer.

The Service layer, including SharePriceService and ChartService, contains the core business logic such as managing stock comparisons and preparing chart data.

The Data layer consists of the Repository for storing price history and the external Provider (Yahoo API) for retrieving stock data.

This improves modularity, making the system easier to test, maintain, and extend.

Secondly, the system adopts the Model–View–Controller (MVC) pattern.

The Model is represented by domain entities such as Stock, Comparison, and PriceHistory, which encapsulate the core data and rules of the system.

The View corresponds to the UI components that present data to the user, including charts and input forms.

The Controller acts as an intermediary that handles user actions and invokes the appropriate services.

This ensures that changes to the user interface do not impact business logic, supporting flexibility and scalability.

The Adapter Pattern is used to integrate the external Yahoo Finance API through the Provider component.

This component acts as an abstraction layer that converts external API responses into the system's internal data structures. By isolating the external dependency, the system remains robust against API changes and allows alternative data providers to be introduced with minimal impact on the rest of the system.

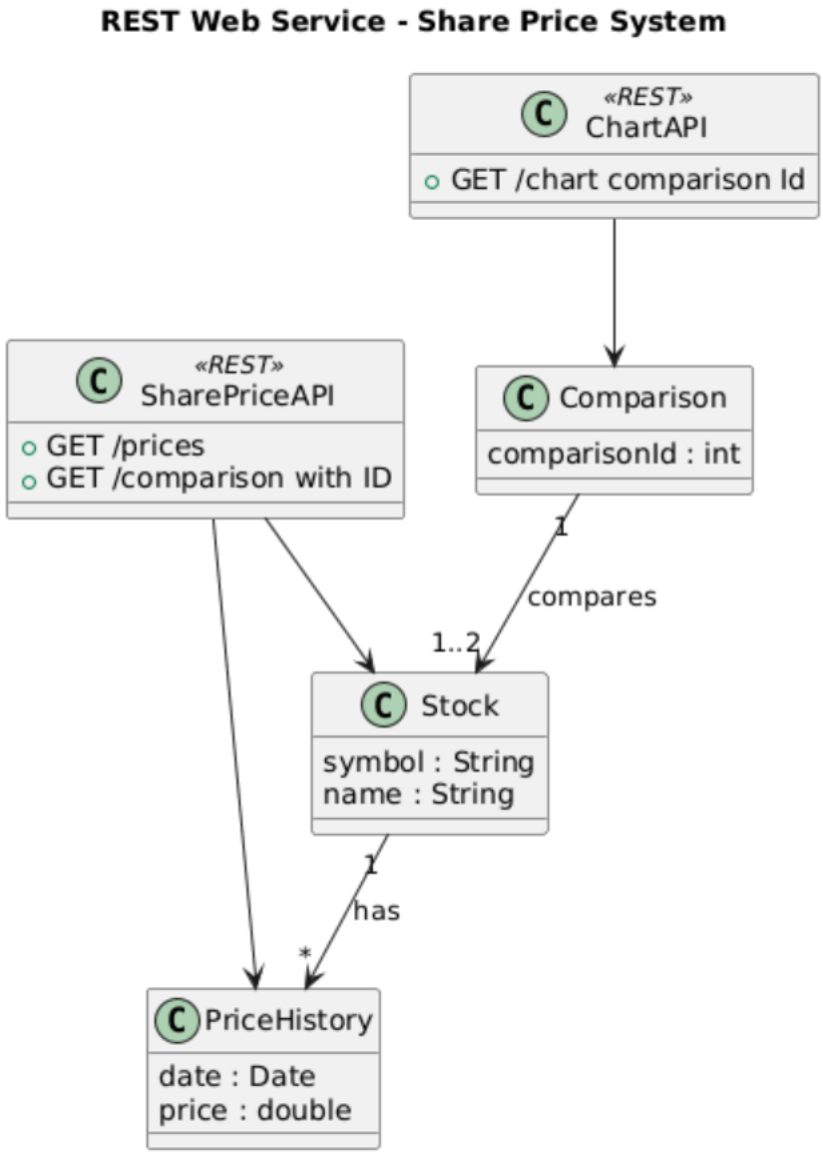
The system demonstrates a Pipes and Filters architecture in its data processing flow. Stock data passes through a sequence of stages: it is retrieved from the external provider, processed within the service layer, optionally stored in the repository, and then transformed by the chart service for presentation.

Each stage performs a specific transformation, promoting reusability and simplifying debugging or extension of the data pipeline.

The system aligns with an N-tier Architecture, separating it into client, server, and data tiers. The client tier consists of the frontend UI components, the server tier contains the application logic and services, and the data tier includes both the internal database and external APIs. This supports independent deployment, improves scalability, and enhances system security.

Software Architecture and Design Coursework Sprint 3: Share price

Abbas Mohamed – A00041006
Keeley Sheridan – E23598145



Abbas Mohamed – A00041006
Keeley Sheridan – E23598145

Application of Service-Oriented Architecture (SOA) Principles

The Share Price Comparison System applies key Service-Oriented Architecture (SOA) principles by structuring core functionality into independent, reusable services with well-defined boundaries and interfaces.

Service Boundaries

The system defines two primary services: SharePriceService and ChartService, each responsible for a distinct area of functionality.

- **The SharePriceService** handles business logic related to retrieving stock data, managing comparisons, and coordinating access to the data layer (repository and external provider).
- **The ChartService** is responsible for transforming processed stock data into a format suitable for visualization in the user interface.

By separating these responsibilities, each service can evolve independently without affecting others.

Loose Coupling through Interfaces

SOA promotes loose coupling by ensuring that services interact through well-defined interfaces rather than direct dependencies. In this system, services expose methods (for ex: retrieving price history or generating chart data) that can be consumed without needing to understand their internal implementation.

Interoperability

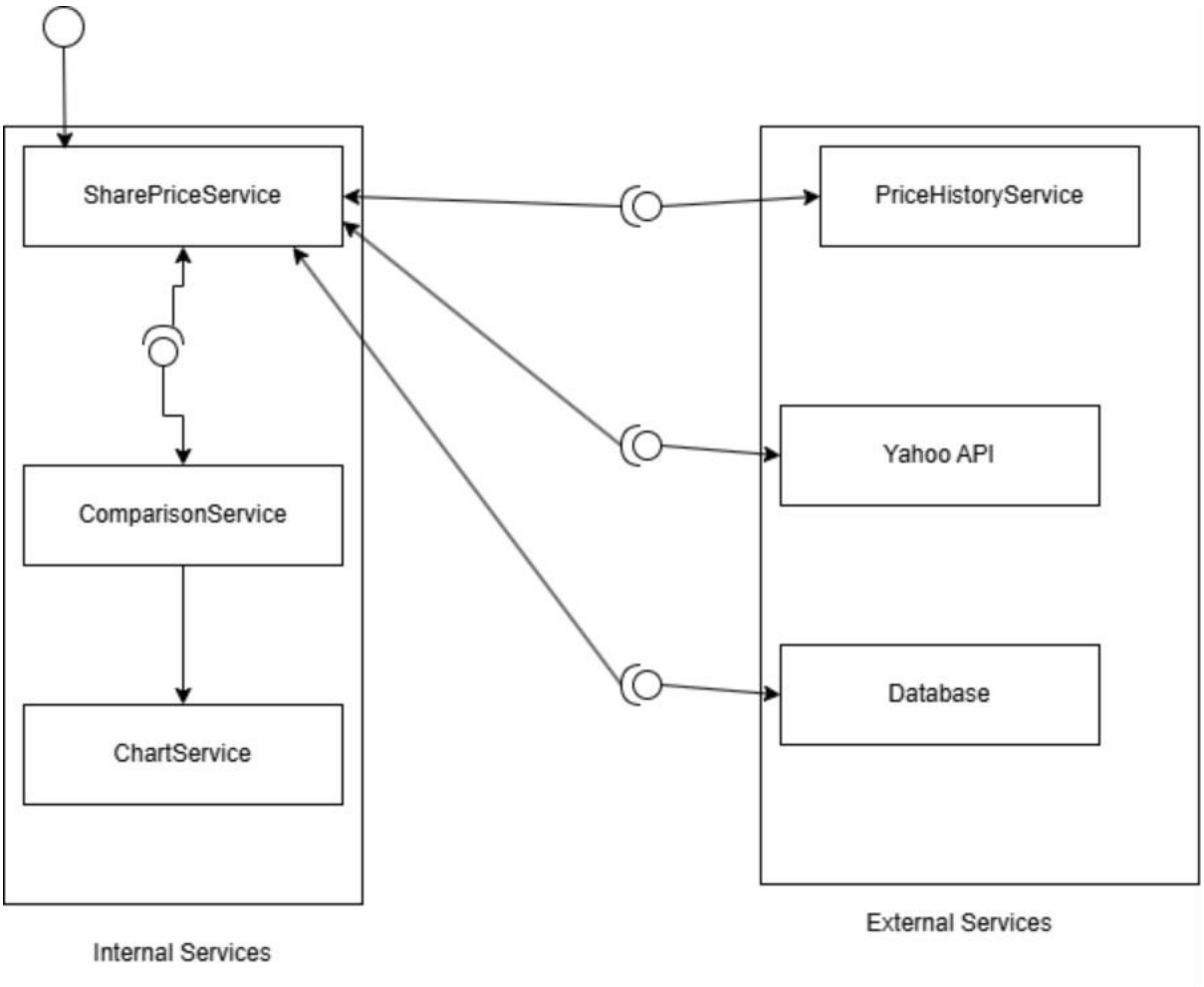
The system supports interoperability by exposing its services in a way that can be accessed by different types of clients. For example, the SharePriceService could be implemented as a REST API endpoint allowing web, mobile, or third-party applications to retrieve stock data.

Because these services communicate using widely accepted protocols and data formats, they are platform-independent and can be consumed by any client capable of making HTTP requests.

By defining clear service boundaries, enforcing interaction through interfaces, and enabling interoperability via standard communication protocols, the system applies SOA principles.

Software Architecture and Design Coursework Sprint 3: Share price
Abbas Mohamed – A00041006
Keeley Sheridan – E23598145

SOA Diagram

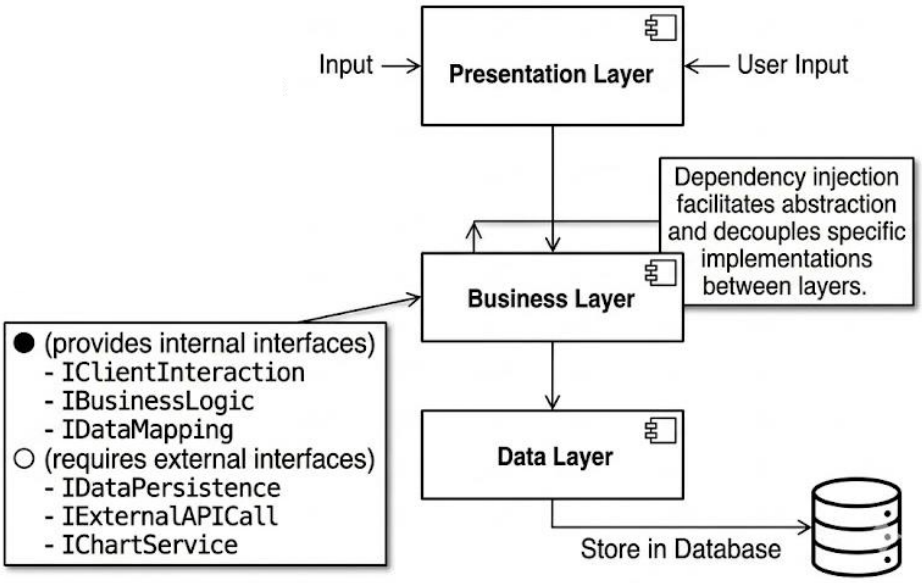


Services Identified

- SharePriceService (core logic)
- Data Service (API access)
- Storage Service (data persistence)
- Chart Service (visualisation)

Software Architecture and Design Coursework Sprint 3: Share price
Abbas Mohamed – A00041006
Keeley Sheridan – E23598145

Layered Architecture



Structure

- Presentation Layer
- Business Layer
- Data Layer

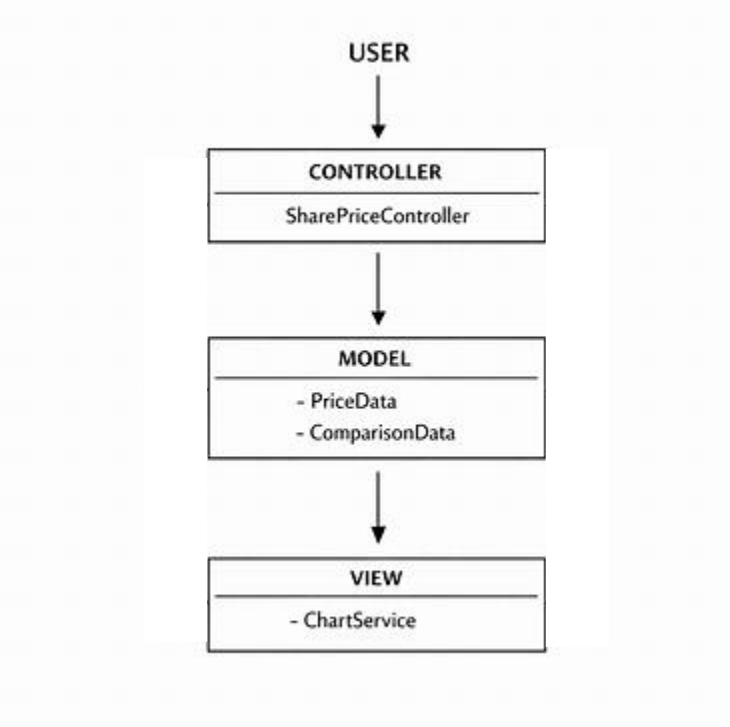
Implementation

- **Controller → Service → Data**

Benefit

- **Separation of concerns**
- **Easier maintenance**

Model View Controller (MVC) Diagram



MVC Role	**Implementation**
Controller	SharePriceController
Model	PriceData, ComparisonData
View	ChartService

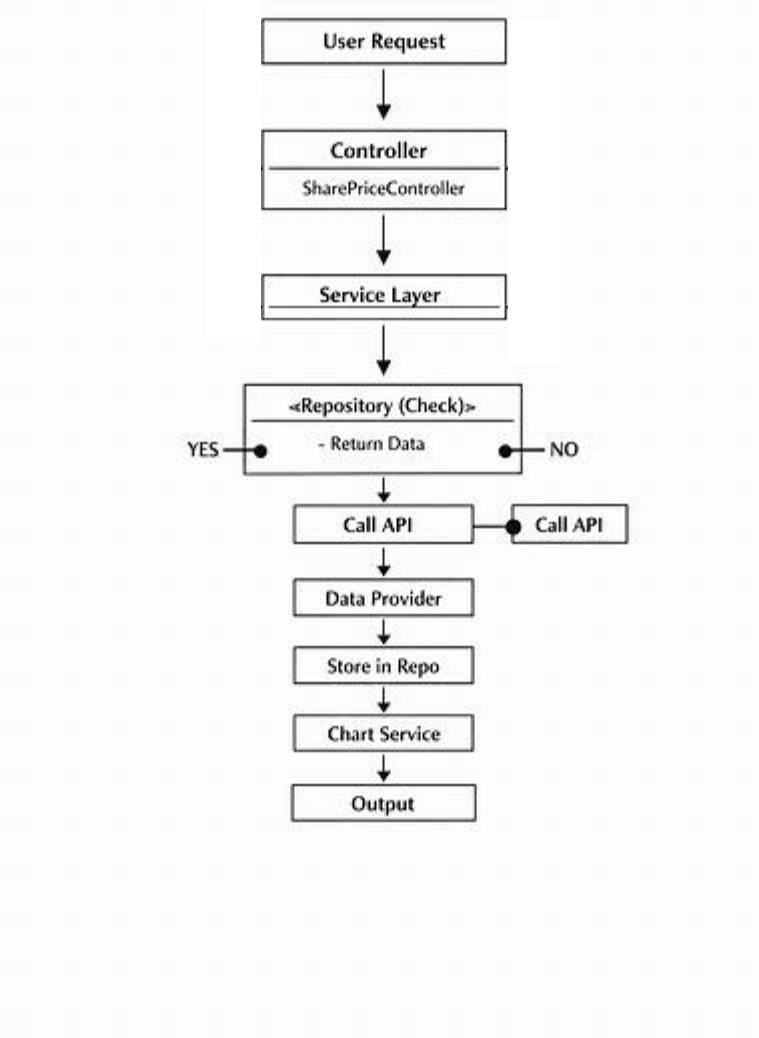
Benefit

- Separates UI from logic
- Improves flexibility

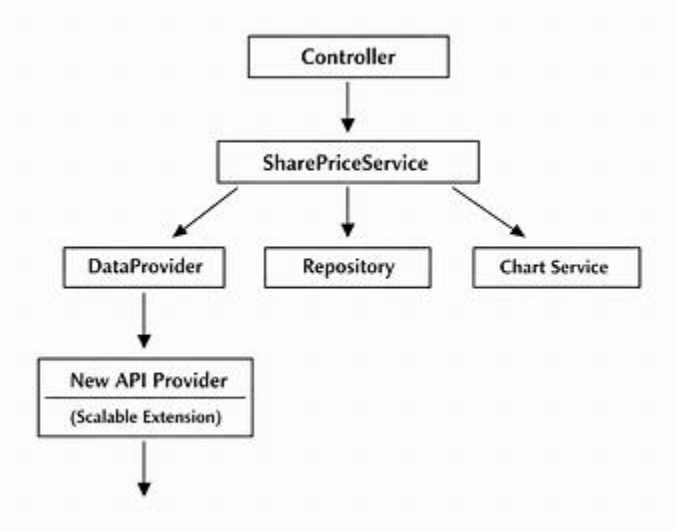
Software Architecture and Design Coursework Sprint 3: Share price

Abbas Mohamed – A00041006
Keeley Sheridan – E23598145

System Flow Diagram



Reusability & Scalability Diagram



Reusability

Achieved through

- Interface-based design
- Modular services
- Separation of concerns

Example

- **ChartService reusable in other systems**

Scalability

Achieved through

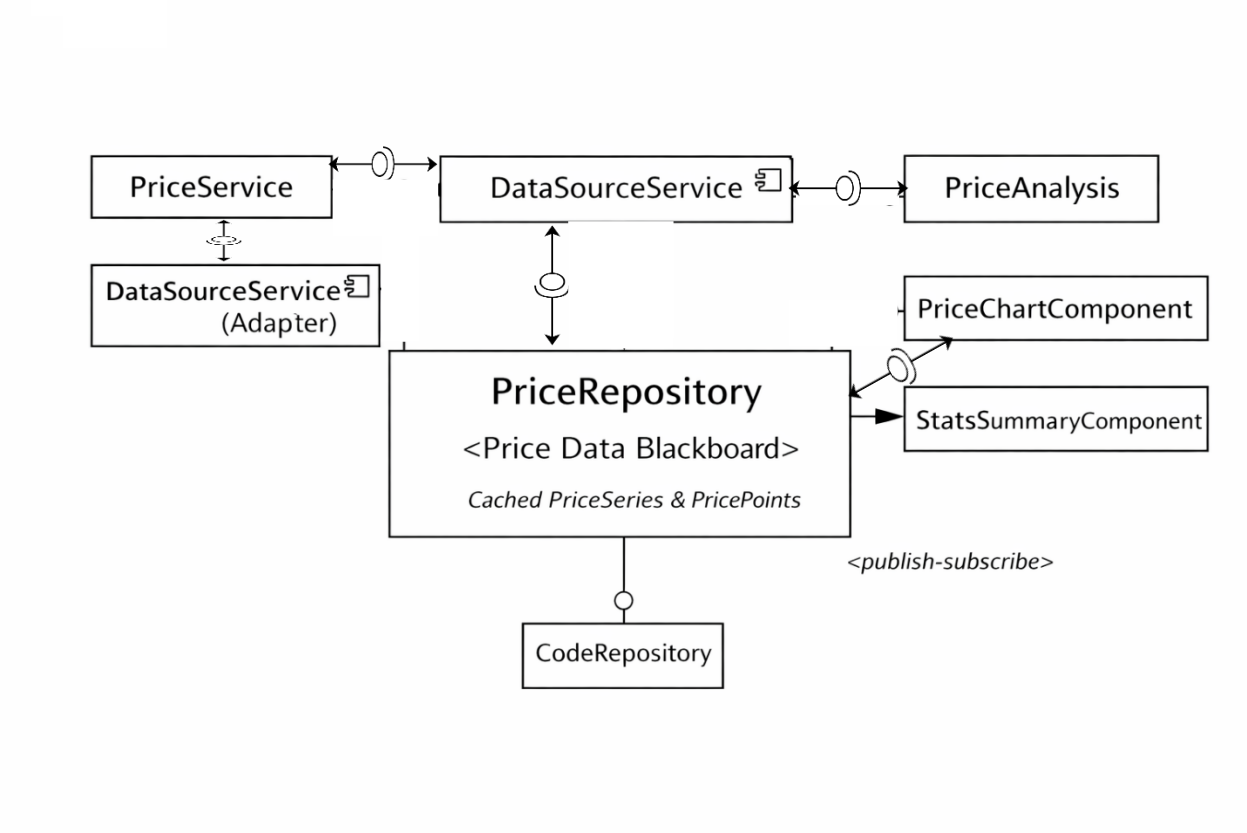
- Service-oriented structure
- Replaceable components
- Layered architecture

Justification

- The architecture supports future growth while maintaining system stability.

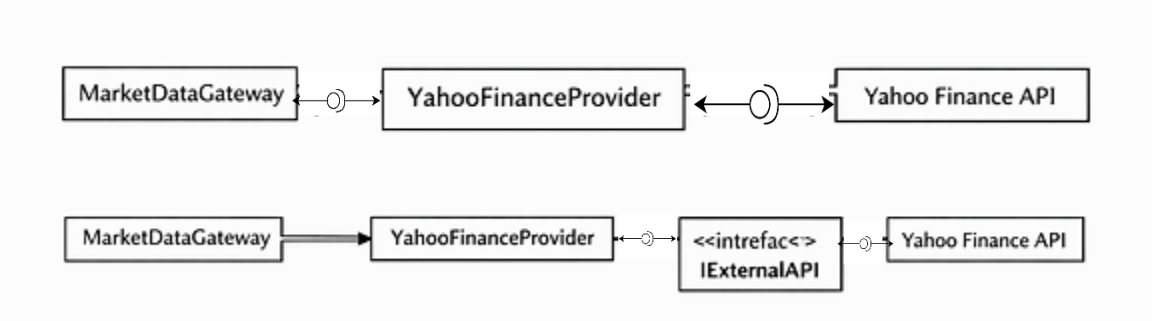
Software Architecture and Design Coursework Sprint 3: Share price
Abbas Mohamed – A00041006
Keeley Sheridan – E23598145

Blackboard



- PriceRepository → Blackboard (shared state)
- DataSourceService → Adapter (external data integration)
- PriceService → Knowledge source (fetches and updates Blackboard)
- PriceAnalysis → Knowledge source (derives insights from Blackboard)
- UI components → Viewers (subscribe to Blackboard updates)

Adapter Pattern



Applied in

- YahooFinanceProvider

Purpose

- Wrap external API
- Allow easy replacement

Justification

These styles ensure:

- Maintainability
- Flexibility
- Scalability

Software Architecture and Design Coursework Sprint 3: Share price
Abbas Mohamed – A00041006
Keeley Sheridan – E23598145

Test Case	Input	Expected Output	Result
1. Retrieve Share Prices	Symbol: AAPL Date Range: Last 5 days	PriceData object returned Data stored if not already present	PASS
2. Retrieve Stored Data (Offline Mode)	Symbol: AAPL (already stored)	Data loaded from repository No API call made	PASS
3. Compare Two Shares	Symbols: AAPL, MSFT	ComparisonData object returned Both datasets retrieved successfully	PASS
4. Generate Chart	Valid PriceData	ChartData object returned Chart title and values correctly generated	PASS
5. Invalid Input Handling	Empty symbol	Error or null handled gracefully	PASS

Testing focused on

- Validating system behaviour
- Ensuring correct interaction between components
- Confirming outputs match expected results